

Generación autónoma de microservicios

Dónde está la industria · cómo funciona · qué cuesta · hacia dónde vamos

Ingeniería AgentIA

2026

Dónde está la industria: tres caminos

1. Agentes generalistas de código

Arreglan o completan código *ya existente*.

- Dominan los rankings.
- Saturados y **caros** (\$5–\$30 por tarea).

2. Generadores guiados por especificación

Construyen la aplicación *desde una especificación formal*.

- **Es nuestro camino.**
- Solo se resuelve **12–55%**: mucho espacio.

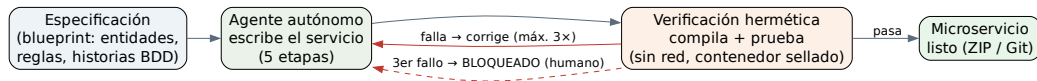
3. Agentes que se auto-mejoran

Escriben su propio “manual” de trabajo.

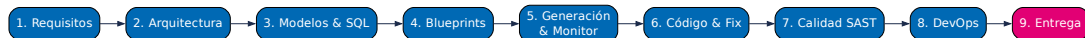
- Prometedor en laboratorio.
- En código real, mejora **casi nula**.

La lección de toda la investigación: el ingrediente que gana es una *verificación* de fiar, no un modelo más listo.

Cómo funciona la solución



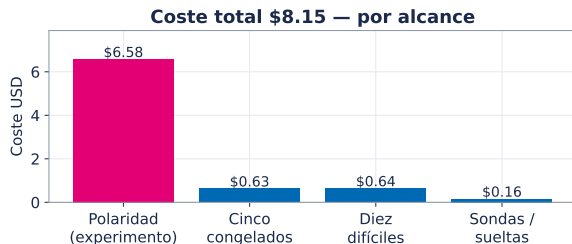
El proceso, de punta a punta



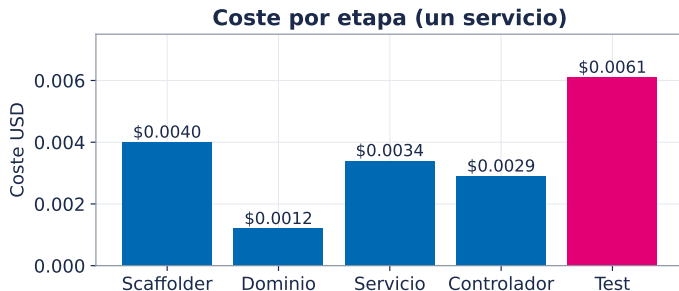
Nueve etapas: de la especificación a la entrega (ZIP o Git), con generación y verificación en el centro.

- **Se escribe** un servicio completo, no fragmentos: entidades, lógica, API, tests.
- **Se verifica** en un contenedor sellado, sin red: compila y pasa sus pruebas, o no se entrega.
- **Se autocorrigie** hasta 3 veces; al tercer fallo se detiene y pide a un humano. Nunca un bucle infinito, nunca un “éxito” no comprobado.
- **Se audita** el cumplimiento de las reglas del proyecto, aplicadas por la plataforma, no por la opinión del modelo.

El coste (medido, sistema de registro + MLflow)

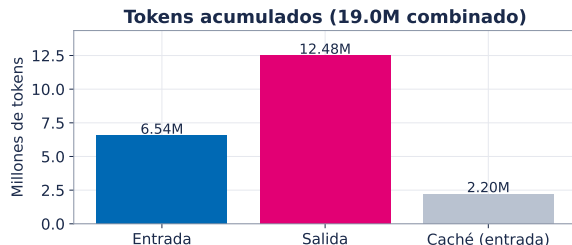


- Un microservicio (dificultad media, recién ejecutado): **\$0.067**
- Rango por servicio: \$0.052 – \$0.077
- Un servicio **garantizado** (se generan 3, se entrega el que compila): **\$0.20**
- **Total acumulado: \$8.15** en 1,219 llamadas



El test es la etapa más cara (\$0.0061); dominio la más barata (\$0.0012). El coste escala con el tamaño del servicio, de forma predecible.

Tokens (último reporte de MLflow)



- **19.0M tokens combinados** (registro)
- 6.5M entrada · 12.5M salida · 2.2M desde caché
- **≈78K tokens por servicio**
- ≈15.6K por llamada; salida ≈1.9× la entrada

- **La plataforma ya verifica su propia salida.** Antes reportaba “0 de 15 verificadas”; el bloqueo era un defecto de un carácter.
- **Ambas vías de producto compilan y prueban,** no solo una.
- **La métrica de calidad dejó de ser manipulable.**
- **El registro de coste es honesto** y visible en MLflow.
- **Queda un punto abierto:** el verificador a veces cambia su veredicto (se corrige en la siguiente iteración).

La idea

Que el sistema aprenda de sus propios errores y mejore su “manual” de generación automáticamente.

Lo que la evidencia exige

Un sistema que aprende de un juez ruidoso aprende ruido. Por eso el orden correcto es:

- 1 Hacer **sólido el verificador**.
- 2 Hacer que la **medida detecte** los fallos reales.
- 3 *Solo entonces*, encender el bucle de auto-mejora.

El mecanismo de auto-mejora **ya está construido**; se enciende cuando la base sea fiable.

Paso	Qué	Esfuerzo
1	Sólido el verificador (una configuración)	corto
2	Detectar fallos reales en la medida	corto
3	Medir si un “manual” aporta	\$3–\$5
4	Encender la auto-mejora (ya construida)	condicionado

El mensaje

De “no podía verificar su propia salida” a “genera, compila y verifica por \$0.07, y se auto-mejorará cuando la base sea fiable”.